BCA 5th Semester — .NET Technology

Internal Exam Preparation Guide

Exam Sections	MCQ (10 marks) + Short Q (12 x 5) + Long Q (8 x 10)
Total Marks	10 + 60 + 80 = 150 marks
Units Covered	Unit 1 to Unit 4 (C#/.NET Framework)

All questions include model answers. Short Q = 5 marks | Long Q = 10 marks.

SECTION A — Multiple Choice Questions [10 x 1 = 10 Marks]

1. Which component of CLR converts MSIL code to native machine code at runtime?

a) CLS
b) JIT Compiler
c) GC
d) FCL

Correct Answer: b) JIT Compiler

Explanation: The Just-In-Time (JIT) compiler converts Intermediate Language (MSIL) code to native machine code just before execution.

2. Where are value types stored in memory?

a) Heap
b) Register
c) Stack
d) Static memory

Correct Answer: c) Stack

Explanation: Value types (int, struct, enum, etc.) are stored on the stack. Reference types are stored on the heap.

3. Which keyword is used to declare a method that can be overridden in a derived class?

a) abstract
b) override
c) virtual
d) sealed

Correct Answer: c) virtual

Explanation: The "virtual" keyword in the base class marks a method as overridable. The derived class uses "override" to provide a new implementation.

4. What does ExecuteNonQuery() return in ADO.NET?

a) A DataSet
b) First column of first row
c) Number of rows affected
d) A SqlDataReader

Correct Answer: c) Number of rows affected

Explanation: ExecuteNonQuery() is used for INSERT, UPDATE, DELETE statements and returns the number of rows affected. It does not return data rows.

5. Which of the following is NOT a valid type of constructor in C#?

a) Static constructor
b) Copy constructor
c) Virtual constructor
d) Private constructor

Correct Answer: c) Virtual constructor

Explanation: C# does not support virtual constructors. Constructors cannot be marked with the virtual keyword.

6. What is the output of Func add = (a, b) => a + b; Console.WriteLine(add(3, 4));?

a) 34
b) 7
c) Compile error
d) 0

Correct Answer: b) 7

Explanation: Func takes two int inputs and returns an int. The lambda (a,b) => a+b adds them. add(3,4) = 7.

7. Which generation of the Generational GC model holds short-lived objects like temporary variables?

a) Generation 2
b) Generation 1
c) Generation 0
d) Generation 3

Correct Answer: c) Generation 0

Explanation: Generation 0 holds newly created, short-lived objects. Gen 1 holds objects that survived Gen 0. Gen 2 holds long-lived objects.

8. Which LINQ syntax is closest to SQL syntax?

a) Method syntax
b) Query syntax
c) Lambda syntax
d) Extension method syntax

Correct Answer: b) Query syntax

Explanation: Query syntax uses "from...where...select" clauses similar to SQL. Method syntax uses extension methods like .Where() and .Select().

9. What does the "ref" keyword require that "out" does not?

a) The variable must be a reference type
b) The variable must be initialized before passing
c) The variable must be a static field
d) The method must return void

Correct Answer: b) The variable must be initialized before passing

Explanation: "ref" requires the variable to be initialized before being passed. "out" does not require prior initialization — the method is responsible for assigning it.

10. In ASP.NET Web Forms, which state management stores data per user across multiple pages on the server?

a) ViewState
b) Cookies
c) ApplicationState
d) SessionState

Correct Answer: d) SessionState

Explanation: SessionState is server-side, per-user, and accessible across multiple pages for the duration of the user's session. ViewState is per-page and client-side.

SECTION B — Short Questions [12 x 5 = 60 Marks]

Unit 1 — .NET Framework

Q1. What is CLR? Explain its main components. [5 Marks]

Answer:

CLR (Common Language Runtime) is the runtime environment for the .NET Framework. It manages the execution of .NET applications and provides essential services.

Main Components:

1. **JIT Compiler** — Converts MSIL (Intermediate Language) code to native machine code at runtime, optimized for the current platform.
2. **Memory Management (GC)** — Automatically allocates and frees memory. Uses the Garbage Collector (GC) to remove unreachable objects.
3. **Security** — Enforces code access security (CAS) and role-based security policies.
4. **Exception Handling** — Provides a unified, structured way to handle runtime errors through System.Exception and its derived classes (NullReferenceException, IndexOutOfRangeException, etc.).

Q2. Differentiate between managed code and unmanaged code. [5 Marks]

Answer:

The key differences between managed and unmanaged code are:

Managed Code	Unmanaged Code
Compiled to MSIL/IL first	Compiled directly to native code
Runs under CLR supervision	Runs directly under the OS
Automatic GC, Security, Exception handling	Manual memory management
Platform-independent (any CLR)	Platform-specific
Examples: C#, VB.NET, F#	Examples: C, C++

Q3. What is MSIL? Why is it platform-independent? [5 Marks]

Answer:

MSIL (Microsoft Intermediate Language), also called IL or CIL, is a low-level, platform-independent bytecode generated by .NET language compilers (csc.exe for C#).

Why it is platform-independent:

- MSIL is not tied to any specific CPU architecture or OS.
- When an application runs, the CLR's JIT compiler converts MSIL to native machine code specific to the current platform.
- Any platform that has a CLR (or compatible runtime like Mono) can execute the same MSIL, enabling "write once, run anywhere" within .NET.
- MSIL also enables language interoperability — C#, VB.NET, and F# all compile to the same MSIL format.

Q4. Explain the Generational Garbage Collection model (Gen 0, 1, 2) with all four GC phases. [5 Marks]

Answer:

Four Phases of GC:

1. **Tracking** — GC identifies root references (stack variables, static fields, CPU registers). Objects reachable from roots are marked as "reachable".
2. **Marking** — GC traverses from roots and marks all reachable objects. Unreachable objects (garbage) remain unmarked.
3. **Sweeping** — Unmarked (unreachable) objects are removed from memory, creating free gaps.
4. **Compaction** — Remaining live objects are moved together to eliminate fragmentation and reclaim contiguous memory.

Generational Model:

- **Generation 0:** Newly created, short-lived objects (e.g., temp variables). Collected most frequently.
- **Generation 1:** Objects that survived one GC cycle from Gen 0. Buffer between short and long-lived.
- **Generation 2:** Long-lived objects (e.g., static data, application-wide resources). Collected least frequently.

Unit 2 — Language Basics

Q5. Differentiate between value type and reference type with examples. [5 Marks]

Answer:

Value Type	Reference Type
Stores the actual value	Stores a reference (address) to the data
Stored in Stack	Data stored in Heap; reference in Stack
Assignment copies the value	Assignment copies the reference
Changes in copy do NOT affect original	Changes through one reference affect all
int, double, bool, char, struct, enum	class, string, array, interface, delegate

Example:

```
int a = 10; int b = a; b = 20; // a is still 10 (value copy)
```

```
Person p1 = new Person(); Person p2 = p1; p2.Name = "Bob"; // p1.Name is also "Bob"
```

Q6. What is implicit vs explicit type conversion? Give examples of each. [5 Marks]

Answer:

Implicit Conversion: Automatic, no data loss, no special syntax. Smaller to larger compatible type.

```
int x = 10; long y = x; // int -> long (implicit)
```

```
int a = 5; float b = a; // int -> float (implicit)
```

Explicit Conversion (Casting): Manual conversion, possible data loss, requires cast syntax.

```
double d = 9.78; int n = (int)d; // n = 9, decimal lost
```

Using Convert class:

```
string s = "123"; int n = Convert.ToInt32(s); // string -> int
```

Using Parse:

```
int n = int.Parse("456"); // string -> int
```

Q7. What is the difference between the ref and out keywords? [5 Marks]

Answer:

ref	out
Must be initialized before passing	Does NOT need to be initialized
Method can read and modify	Method must assign a value before returning
Used when method needs to update existing value	Used when method returns multiple values

```
void Double(ref int x) { x = x * 2; }
void GetVal(out int x) { x = 100; } // x need not be initialized
```

Q8. What are jagged arrays? How do they differ from multi-dimensional arrays? [5 Marks]

Answer:

Jagged Array: An array of arrays where each sub-array can have a different length. Also called "array of arrays".

```
int[][] jagged = new int[3][];
jagged[0] = new int[] { 1, 2 };
jagged[1] = new int[] { 3, 4, 5 }; // different lengths!
jagged[2] = new int[] { 6 };
```

Multi-Dimensional Array: A rectangular grid where all rows have the same number of columns.

```
int[,] matrix = new int[3, 4]; // 3 rows, 4 columns each
```

Key Difference: Jagged arrays allow variable row sizes; multi-dimensional arrays are always rectangular.

Unit 3 — Creating Types

Q9. List the types of constructors in C# with a brief note on each. [5 Marks]

Answer:

1. **Default Constructor:** No parameters. Initializes fields with default values. Can be built-in or user-defined.
2. **Parameterized Constructor:** Takes parameters to initialize fields at object creation time.
3. **Copy Constructor:** Takes an object of the same class as parameter. Creates a new object copying values from existing.
4. **Static Constructor:** Marked with "static". Called once by CLR before first use. No parameters or access modifier. Used to initialize static fields.
5. **Private Constructor:** Prevents external instantiation. Used in the Singleton pattern.
6. **Constructor Chaining:** One constructor calls another using "this()" or "base()" keyword.

Q10. Differentiate between abstract class and interface. [5 Marks]

Answer:

Abstract Class	Interface
Can have both abstract AND concrete methods	Only method/property signatures (no implementation)
Can have member fields and constructors	No fields, no constructors
Can have access modifiers on members	All members are implicitly public
Single inheritance only	A class can implement multiple interfaces

Use keyword: abstract	Use keyword: interface
Used for "is-a" relationship with shared code	Used to define a contract/capability

Q11. What is method overloading? How does it differ from method overriding? [5 Marks]

Answer:

Method Overloading (Compile-Time Polymorphism): Multiple methods with the same name but different signatures (different number, type, or order of parameters) within the same class. The compiler resolves which to call.

```
int Add(int a, int b) { return a + b; }
```

```
double Add(double a, double b) { return a + b; } // overloaded
```

Method Overriding (Runtime Polymorphism): A derived class provides its own implementation of a base class method marked as "virtual" or "abstract". Uses "override" keyword.

```
class Animal { public virtual void Speak() { ... } }
```

```
class Dog : Animal { public override void Speak() { ... } }
```

Overloading	Overriding
Same class	Parent-child (inheritance) relationship
Resolved at compile time	Resolved at runtime
Different signatures	Same signature

Unit 4 — Advanced C#

Q12. What is a delegate in C#? How is it different from an event? [5 Marks]

Answer:

Delegate: A type-safe function pointer that holds a reference to a method with a matching signature. Can hold normal, static, or instance methods.

```
delegate int Operation(int a, int b);
```

```
Operation op = Add; int result = op(5, 3); // result = 8
```

Event: A special member of a class that wraps a delegate and restricts who can invoke it. Only the declaring class can fire (invoke) the event; outside classes can only subscribe (+=) or unsubscribe (-=).

Delegate	Event
Any code can invoke it directly	Only the declaring class can invoke it
More flexible but less encapsulated	Enforces publisher-subscriber pattern
Foundation for events	Built on top of delegates

SECTION C — Long Questions [8 x 10 = 80 Marks]

Unit 1

L1. Explain the complete execution process of a .NET application — from C# source code to native machine code running on the OS. Include the role of CLS, FCL, BCL, MSIL, CLR, and JIT compiler. [10 Marks]

Answer:

Step-by-Step .NET Execution Process:

Step 1 — Source Code: The developer writes code in a .NET language (C#, VB.NET, F#). Each language has its own compiler (csc.exe for C#, vbc.exe for VB.NET).

Step 2 — Compilation to MSIL: The language compiler compiles the source code into MSIL (Microsoft Intermediate Language), also called IL or CIL. This is NOT native machine code — it is CPU-agnostic bytecode. The compiler also produces Metadata describing types, methods, and references. These are packaged into an Assembly (.exe or .dll).

Step 3 — CLR Loads the Assembly: When you run the application, the CLR (Common Language Runtime) loads the assembly. CLR reads the IL code and metadata.

Step 4 — JIT Compilation: The CLR's JIT (Just-In-Time) compiler converts MSIL to native machine code specific to the current platform (x86/x64, etc.). This happens at runtime, just before each method is first executed. JIT optimizes code for the specific CPU.

Step 5 — Execution with CLR Services: The native code runs under CLR supervision. CLR provides: Garbage Collection (memory management), Security enforcement, Exception handling (System.Exception hierarchy), Thread management.

Supporting Components:

- **CLS (Common Language Specification):** A set of rules ensuring language interoperability. All .NET languages must comply with CLS so code written in C# can interact with VB.NET code.
- **FCL (Framework Class Library):** A large collection of pre-built reusable classes covering I/O, networking, collections, threading, etc.
- **BCL (Base Class Library):** A subset of FCL — the core foundational classes like System, System.IO, System.Collections.

Execution Flow Diagram:

C# Source File (.cs)
C# Compiler (csc.exe)
Assembly: MSIL + Metadata (.exe / .dll)
CLR loads Assembly
JIT Compiler -> Native Machine Code
Operating System executes native code

Unit 1

L2. What is Garbage Collection in .NET? Explain all four phases of GC (Tracking, Marking, Sweeping, Compaction) and the Generational GC model with suitable diagrams/examples. [10 Marks]

Answer:

Garbage Collection (GC) is a feature of CLR that automatically manages memory. It frees memory occupied by objects that are no longer referenced by any live part of the program.

Phase 1 — Tracking Object References:

GC identifies "roots" — references that are definitely alive. Roots include: local variables on the stack, static/global variables, CPU registers. Objects reachable from roots are "reachable"; others are candidates for collection.

Phase 2 — Marking Phase:

GC traverses the object graph from roots and marks every reachable object. Example: Root -> A -> C -> D (marked). B and E are not reachable from any root (unmarked = garbage).

Phase 3 — Sweeping Phase:

Unmarked objects are swept (removed) from memory. Before: [A][B][C][D][E]. After sweeping: [A][ ][C][D][ ] — gaps remain.

Phase 4 — Compaction Phase:

Live objects are moved together to eliminate fragmentation. After: [A][C][D][ ][ ] — free memory is now contiguous at the end, available for new allocations.

Generational GC Model:

The GC divides the managed heap into 3 generations to improve efficiency:

Generation	Description	Collection Frequency
Generation 0	Newly created short-lived objects (temp variables, local objects)	Most frequent
Generation 1	Objects that survived one Gen 0 collection. Buffer between short-lived and long-lived.	Less frequent
Generation 2	Long-lived objects: static data, application-wide resources, large objects.	Least frequent

Benefit: Instead of scanning all objects every time, GC focuses on Gen 0 (newest, most likely garbage) most often, making collection efficient.

Unit 2

L3. Explain the C# data type hierarchy. Cover value types (integral, floating, char, enum, struct) and reference types (object, string, dynamic, class, array). Include storage mechanism (stack/heap) and floating-point precision differences. [10 Marks]

Answer:

C# Data Type Classification:

1. Value Types — Stored directly on the stack. Assignment copies the actual value.

a) Integral Types:

Signed: sbyte (8-bit), short (16-bit), int (32-bit), long (64-bit) — hold positive and negative numbers.

Unsigned: byte (8-bit), ushort (16-bit), uint (32-bit), ulong (64-bit) — hold only non-negative numbers.

b) Floating-Point Types:

Type	Size	Precision	Use Case
float	4 bytes (32-bit)	6-9 significant digits	Graphics, sensors
double	8 bytes (64-bit)	15-16 significant digits	General math (default)
decimal	16 bytes (128-bit)	28-29 significant digits	Financial calculations

c) char: 16-bit Unicode character. Stores a single character (e.g., 'A', 'z').

d) bool: true or false. 1-bit logical value.

e) enum: User-defined set of named constants. Default underlying type is int.

f) struct: User-defined value type grouping related data. Stored on stack; no inheritance.

2. Reference Types — Reference (address) stored on stack; actual data on the heap.

object: Base type of all types in C#. Statically typed. Must cast before accessing members.

string: Immutable sequence of Unicode characters. System.String class. Supports concatenation, interpolation, Substring, Split, Replace, etc.

dynamic: Type resolved at runtime. Bypasses compile-time type checking. Flexible but risky.

class: User-defined reference type. Supports inheritance, polymorphism, encapsulation.

array: Reference type. Fixed-size collection of same-type elements. Single, multi-dimensional, and jagged arrays are supported.

Unit 2

L4. What is a struct in C#? Explain with stack/heap diagrams how value copying works when a struct contains value-type fields and reference-type fields (shallow copy behavior). [10 Marks]

Answer:

Struct: A user-defined value type in C# used to group related data. Unlike classes, structs are stored on the stack, are copied by value, do not support inheritance, and cannot have a user-defined parameterless constructor.

```
struct Point { public int X; public char Label; }
```

Case 1: Struct with all value-type fields

```
Point p1; p1.X = 10; p1.Label = 'A';
```

```
Point p2 = p1; // full independent copy
```

```
p2.X = 20; // p1.X is still 10!
```

Since all fields are value types, a complete independent copy is made. Modifying p2 does NOT affect p1.

Case 2: Struct with a reference-type field (shallow copy)

```
struct Person { public int X; public string Name; }
```

```
Person p1 = new Person { X=10, Name="John" };
```

```
Person p2 = p1; // shallow copy
```

When copied: X (int) gets a full copy. Name (string reference) — BOTH p1 and p2 point to the SAME string object "John" on the heap.

```
p2.Name = "Mike"; // creates NEW string on heap; p1.Name still "John"
```

Note: Strings are immutable, so reassigning creates a new object. But with a mutable reference type (class), changes through p2 would affect p1 as well.

Key Takeaways:

- Structs are always copied by value.
- Value-type fields are fully independent after copy.
- Reference-type fields share the same object until one is reassigned.
- Modifying the shared object affects all structs holding that reference.

Unit 3

L5. Explain the four OOP principles in C# — Encapsulation, Inheritance, Polymorphism, and Abstraction — with code examples. Also list the types of inheritance supported in C# and give examples. [10 Marks]

Answer:

1. Encapsulation: Bundling data (fields) and methods that operate on that data within a single unit (class), restricting direct access using access modifiers.

```
class BankAccount {
private double balance; // hidden
public void Deposit(double amt) { balance += amt; }
public double GetBalance() { return balance; } }
```

2. Inheritance: A class (child) acquires properties and behaviors of another class (parent). Promotes code reuse.

```
class Animal { public void Eat() { Console.WriteLine("Eating"); } }
class Dog : Animal { public void Bark() { Console.WriteLine("Woof!"); } }
```

Types of Inheritance in C#:

- **Single:** One child from one parent. class Dog : Animal { }
- **Multilevel:** Chain. class Animal { } -> class Dog : Animal { } -> class Puppy : Dog { }
- **Hierarchical:** Multiple children from same parent. class Dog : Animal { } class Cat : Animal { }
- **Multiple (via Interface):** C# does not support multiple class inheritance. Uses interfaces: class Dog : IAnimal, IPet { }

3. Polymorphism: Same method name, different behavior based on object type.

- **Overloading (compile-time):** Same method name, different signatures in same class.
- **Overriding (runtime):** Derived class overrides base class virtual/abstract method using override keyword.

```
class Animal { public virtual void Sound() { Console.Write("..."); } }
class Dog : Animal { public override void Sound() { Console.Write("Woof"); } }
```

4. Abstraction: Hiding implementation details and showing only the essential interface.

```
abstract class Shape { public abstract double Area(); } // no implementation
class Circle : Shape { public override double Area() { return 3.14 * r * r; } }
```

Interfaces are another form of abstraction — they define a contract with no implementation.

Unit 3

L6. What are properties and indexers in C#? Explain auto-implemented properties, read-only properties, and how indexers allow objects to be accessed like arrays. Provide code examples. [10 Marks]

Answer:

Properties: Special members that provide a flexible mechanism to read, write, or compute private field values. They use get and set accessors.

```
class Person {
private string name; // private backing field
public string Name { // property
get { return name; }
set { if (!string.IsNullOrEmpty(value)) name = value; } } }
```

Auto-Implemented Properties: When no special logic is needed, C# automatically creates a hidden backing field.

```
class Student { public string Name { get; set; } public int Age { get; set; } }
```

Read-Only Property: Set accessor is private — only the class itself can set the value.

```
public string Name { get; private set; } // read-only from outside
```

Indexers: Allow an object to be accessed using array-like syntax (obj[index]). Defined using the "this" keyword.

```
class StudentList {
private string[] names = new string[5];
public string this[int index] {
get { return names[index]; }
set { names[index] = value; } } }
```

Usage:

```
StudentList list = new StudentList();
list[0] = "Alice"; // uses set accessor
Console.WriteLine(list[0]); // uses get accessor -> "Alice"
```

Key Points: A class can have multiple indexers if they differ in parameter type. Indexers hide internal data structures and enable custom collection classes.

Unit 4

L7. What are delegates in C#? Explain single-cast, multicast, and anonymous delegates. Also explain Func, Action, and Predicate built-in delegate types with lambda expression examples. [10 Marks]

Answer:

Delegate: A type-safe reference to one or more methods. Acts like a function pointer. Enables callbacks and event-driven programming.

```
delegate int Operation(int a, int b); // delegate definition
```

1. Single-Cast Delegate: Holds reference to exactly one method.

```
static int Add(int a, int b) { return a + b; }
```

```
Operation op = Add; Console.WriteLine(op(5, 3)); // 8
```

2. Multicast Delegate: Holds references to multiple methods. Uses += to add and -= to remove. Calls all methods in sequence.

```
delegate void Notify(string msg);
```

```
Notify n = SendEmail;
```

```
n += SendSMS; // multicast: adds second method
```

```
n += SendPush;
```

```
n("Server down!"); // calls all three methods
```

3. Anonymous Delegate (C# 2.0): Inline method without a name using delegate keyword.

```
Action greet = delegate(string name) {
```

```
Console.WriteLine("Hello " + name); };
```

```
greet("Riya");
```

Lambda Expressions (C# 3.0): Concise syntax for anonymous functions using => operator.

Func: Takes input(s), returns a value.

```
Func add = (a, b) => a + b;
```

```
Console.WriteLine(add(4, 6)); // 10
```

Action: Takes input(s), returns void.

```
Action print = msg => Console.WriteLine(msg);
```

```
print("Hello World!");
```

Predicate: Takes one input, always returns bool.

```
Predicate isEven = n => n % 2 == 0;
```

```
Console.WriteLine(isEven(4)); // True
```

```
Console.WriteLine(isEven(7)); // False
```

Unit 4

L8. Explain the ADO.NET architecture. Compare SqlDataReader vs SqlDataAdapter. Write code examples for: (a) SELECT using ExecuteReader(), (b) INSERT using ExecuteNonQuery(), (c) SELECT using DataAdapter.Fill(). [10 Marks]

Answer:

ADO.NET Overview: ADO.NET is the data access framework in .NET for interacting with databases. It supports both connected (real-time) and disconnected (offline) models.

Key Components:

- **Connection:** Establishes a connection to the database (SqlConnection, NpgsqlConnection, MySqlConnection).
- **Command:** Executes SQL statements against the database (ExecuteReader, ExecuteNonQuery, ExecuteScalar).
- **DataReader:** Forward-only, read-only, connected stream of rows. Fast and efficient.
- **DataAdapter:** Bridge between DataSet (offline) and database. Uses Fill() to load and Update() to save.
- **DataSet:** Disconnected, in-memory representation of data (can hold multiple DataTables).

a) SELECT using ExecuteReader():

```
string conn = "Server=localhost;Database=MyDB;Integrated Security=True";
using (SqlConnection cn = new SqlConnection(conn)) {
cn.Open();
SqlCommand cmd = new SqlCommand("SELECT Name,Price FROM Product WHERE InActive=0", cn);
SqlDataReader rdr = cmd.ExecuteReader();
while (rdr.Read()) {
Console.WriteLine(rdr["Name"] + " - " + rdr["Price"]); } }
```

b) INSERT using ExecuteNonQuery():

```
using (SqlConnection cn = new SqlConnection(conn)) {
cn.Open();
string sql = "INSERT INTO Product(Name,Price,Category) VALUES(@N,@P,@C)";
SqlCommand cmd = new SqlCommand(sql, cn);
cmd.Parameters.AddWithValue("@N", "Laptop");
cmd.Parameters.AddWithValue("@P", 75000);
cmd.Parameters.AddWithValue("@C", "Electronics");
int rows = cmd.ExecuteNonQuery();
Console.WriteLine("Rows inserted: " + rows); }
```

c) SELECT using DataAdapter.Fill():

```
using (SqlConnection cn = new SqlConnection(conn)) {
SqlDataAdapter da = new SqlDataAdapter("SELECT * FROM Product", cn);
DataSet ds = new DataSet();
da.Fill(ds, "Product");
foreach (DataRow row in ds.Tables["Product"].Rows) {
Console.WriteLine(row["Name"] + " | " + row["Category"]); } }
```

Comparison: SqlDataReader vs DataAdapter/DataSet

SqlDataReader	DataAdapter/DataSet
Connected (needs open connection)	Disconnected (can work offline)
Forward-only, read-only	Full CRUD operations supported
Faster and uses less memory	Slower but more flexible
Cannot go back to previous rows	Can navigate any row anytime
Best for large read-only data	Best for editing and batch updates